

SOMA: Strategic Orchestration and Memory-Augmented System for Vision-Language-Action Model Robustness via In-Context Adaptation

Zhuoran Li^{1,*}, Zhiyang Li^{1,*}, Kaijun Zhou¹, Jinyu Gu^{1,†}

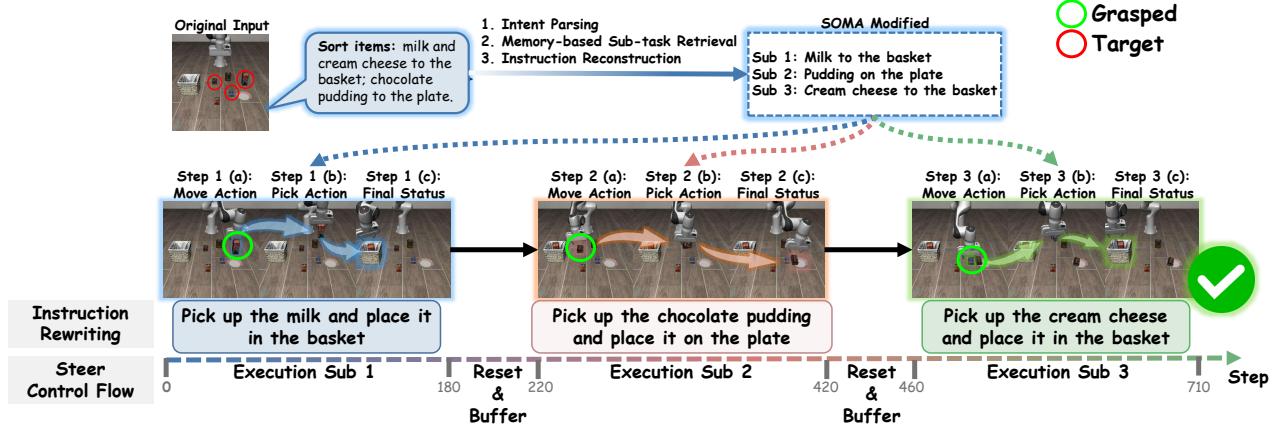


Fig. 1: **Mastering Long-Horizon Manipulation via Memory-Driven Decomposition and Chaining.** SOMA transforms abstract, multi-step instructions into precise execution sequences by orchestrating intent parsing, memory-based retrieval, and adaptive control-flow regulation.

Abstract—Despite the promise of Vision-Language-Action (VLA) models as generalist robotic controllers, their robustness against perceptual noise and environmental variations in out-of-distribution (OOD) tasks remains fundamentally limited by the absence of long-term memory, causal failure attribution, and dynamic intervention capability. To address this, we propose SOMA, a Strategic Orchestration and Memory-Augmented System that upgrades frozen VLA policies for robust in-context adaptation without parameter fine-tuning. Specifically, SOMA operates through an online pipeline of contrastive Dual-Memory Retrieval-Augmented Generation (RAG), an Attribution-Driven Large-Language-Model (LLM) Orchestrator, and extensible Model Context Protocol (MCP) interventions, while an offline Memory Consolidation module continuously distills the execution traces into reliable priors. Experimental evaluations across three backbone models (π_0 , $\pi_{0.5}$, and SmolVLA) on LIBERO-PRO and our proposed LIBERO-SOMA benchmarks demonstrate that SOMA achieves an average absolute success rate gain of 56.6%. This includes a significant absolute improvement of 89.1% in long-horizon task chaining. Project page and source code are available at: <https://github.com/LZY-1021/SOMA>.

I. INTRODUCTION

VLA models have emerged as a leading paradigm for end-to-end robotic manipulation, unifying multimodal perception, linguistic grounding, and continuous motor control within a single learnable policy [1], [2]. In deployment, these

models serve as monolithic sensorimotor controllers that directly translate visual observations and natural language instructions into low-level motor commands.

However, as robotic agents transition from controlled laboratory settings to the unpredictability of open-world environments, the central challenge shifts from in-distribution accuracy to robust in-context adaptation [3]. While traditional fine-tuning is effective in specialized settings, general-purpose robots must operate under interference and noise that cannot be fully anticipated during training. Such disturbances—arising from variations in object appearance, lighting conditions, and linguistic ambiguity—frequently induce OOD perturbations, exposing the structural limitation of static VLA policies and highlighting the need for stronger robustness to environmental noise and causal distractions [4], [5], [6], [7], [8].

Existing approaches each address OOD generalization only partially. Scaling-centric foundation models [9], [10], [11] seek generalization through large data regimes, yet operate as stateless controllers that discard execution history, lacking the **long-term memory** needed to adapt to OOD conditions. Retrieval-augmented frameworks [12], [13] incorporate past demonstrations but suffer from a “success bias”: imitating successful trajectories, they lack the **causal attribution** capability to diagnose failure modes from perceptual discrepancies. Critically, both paradigms treat their knowledge base as a static artifact, allowing early misattributions to persist across tasks. Recent agentic frameworks [14], [15], [16] introduce LLM-based planning,

*Equal contribution.

†Corresponding author: gujinyu@sjtu.edu.cn

¹All authors are with the Institute of Parallel and Distributed Systems (IPADS), Shanghai Jiao Tong University, China. Emails: {airplane, 3390964845, zhoukaijun, gujinyu@sjtu.edu.cn}

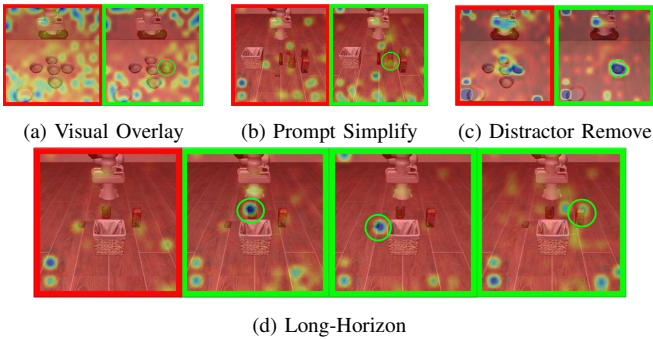


Fig. 2: **Comparing attention maps under varied inputs.** Red and blue indicate high and low attention. Baseline maps (red frames) exhibit diffuse, non-specific focus. Targeted MCP interventions (green frames) facilitate precise object identification and refined edge delineation through: (a) Overlaying visual cues; (b) Simplifying linguistic prompts; (c) Filtering background clutter; and (d) Decomposing long-horizon subtasks.

yet operate as zero-start reasoners with rigid tool integration, lacking the **dynamic orchestration** required to intervene in the perception-action stream.

These observations indicate that achieving robust OOD generalization requires a unified framework that effectively integrates: (i) long-term memory for accumulating and reusing past execution experience, (ii) causal attribution and failure-aware strategic adjustment, (iii) dynamic orchestration of external tools to actively intervene in the perception-action stream, and (iv) a self-evolving consolidation mechanism that continuously refines stored experiences to prevent knowledge stagnation.

To address these limitations, we present **SOMA**, a **Strategic Orchestration and Memory-Augmented** architecture that upgrades frozen VLA policies into robust, adaptive systems without parameter fine-tuning. SOMA operationalizes the aforementioned requirements through a structurally decoupled architecture, featuring three tightly integrated on-line components for task intervention, complemented by an asynchronous offline dual-stage memory consolidation that drives continuous long-term capability improvement.

(i) **Long-Term Experience Alignment:** As shown in Fig. 2, OOD failures often arise from perceptual “attention drift” rather than deficient motor execution: the frozen VLA’s focus is disrupted by environmental or linguistic noise despite its latent capability. To improve robustness against such disturbances, SOMA maintains a **long-term memory bank** of successful and failed executions, enabling it to recall which input-level adjustments (e.g., visual or linguistic) resolved similar prior failures. Via MCP tool interventions [17], SOMA aligns current perceptions with past experience to reuse learned strategies without any parameter updates.

(ii) **Causal Failure Attribution:** For novel failures where simple retrieval is insufficient, SOMA employs a **contrastive dual-memory retrieval** mechanism that retrieves both successes and failures. An LLM analyzes these contrastive pairs to diagnose whether a failure stems from perceptual ambi-

guity or planning deviation, guiding targeted interventions.

(iii) **Dynamic Tool Orchestration:** Diagnostic insights are translated into interventions via the MCP, enabling **dynamic orchestration** of modular tool chains. The LLM composes and invokes MCP sequences based on its causal diagnosis, rectifying visual inputs or refining task context to handle diverse unstructured environments.

(iv) **Memory Consolidation:** After each task, an offline workflow refines the memory bank without interrupting execution. **Initial Diagnosis** attributes the outcome of the latest trajectory and commits the new entry to the Dual-Memory Bank; **Memory Consolidation** then retrieves similar historical experiences and performs cross-task differential analysis, retroactively refining stored attributions and intervention plans. This self-correction loop avoids knowledge stagnation and improves the reliability of future interventions.

Empirical evaluations across the standard **LIBERO-PRO** [7] benchmark and our custom **LIBERO-SOMA** suite demonstrate that our framework consistently enhances in-context adaptation for various VLA models without fine-tuning. On LIBERO-SOMA, where base models often fail under sensory and linguistic noise, SOMA restores performance with an average gain of **59.3%**, notably achieving up to 89.1% absolute improvement in long-horizon tasks. Similarly, on LIBERO-PRO[7], SOMA effectively bridges the distribution gap caused by layout and semantic shifts ($\approx 0\%$ success), yielding a weighted average gain of **54.5%**. These results confirm SOMA as a robust, plug-and-play solution for OOD robotics challenges.

In summary, this work makes three contributions:

- 1) **Attribution-Driven Memory and Extensible Intervention Architecture:** We introduce a closed-loop architecture that moves beyond the “success bias” of conventional RAG. SOMA leverages contrastive dual-memory retrieval to diagnose failure causes and executes targeted interventions via a dynamically orchestrated MCP tool chain, forming a parameter-free, plug-and-play robustification framework for VLA models.
- 2) **Asynchronous System Design:** We propose a decoupled design that separates task-level intervention from long-term knowledge refinement. The online pipeline executes attribution-guided MCP interventions, while offline consolidation performs post-hoc attribution and memory updates to distill reliable priors without interrupting inference or requiring parameter updates.
- 3) **In-Context Adaptation and Experimental Results:** Across diverse tasks and backbones, SOMA significantly improves overall performance and in-context adaptation capabilities. By offloading reasoning to an evolving memory base and extensible tools, SOMA remains robust under OOD conditions where conventional VLAs policies typically fail.

II. RELATED WORK

A. VLA Models and Generalization Challenges

Recent years have witnessed a paradigm shift in robot learning, from specialized policy networks to general-

purpose VLA models. Pioneering works such as π_0 [9] and OpenVLA [10] leverage massive web-scale datasets (e.g., Open-X Embodiment [11]) to enable end-to-end robotic control, integrating visual encoders (e.g., SigLIP [18]) with large language models to translate open-vocabulary instructions into precise continuous actions. However, adapting these models to OOD scenarios via fine-tuning (e.g., X-VLA [19]) incurs high computational costs, motivating the need for parameter-efficient, plug-and-play generalization without weight updates.

B. RAG and In-Context Learning in Robotics

To improve adaptability without fine-tuning, RAG and In-Context Learning (ICL) have been introduced to the robotics domain. Frameworks like RICL [12] and MemER [13] retrieve relevant historical demonstrations via vector databases and condition policy execution by appending them as contextual prompts, enabling adaptation to novel tasks through nearest-neighbor imitation. While effective at leveraging external memory, these methods focus only on successful trajectories and overlook the diagnostic value of failures, highlighting the need for a dual-memory structure that leverages both successes and failures for contrastive guidance.

C. In-Context Adaptive Agentic Frameworks

Beyond passive trajectory retrieval, recent research has explored leveraging LLMs and VLMs as high-level planners with external tool access. Code-as-Policies [14] generates executable Python code for hierarchical tasks, VoxPoser [15] grounds instructions into 3D spatial value maps for motion planning, and MOKA [16] annotates affordances on images via visual prompting. Despite decomposing complex instructions into in-context adaptive executable primitives, these frameworks lack the experiential memory and dynamic correction needed for closed-loop adaptation.

III. METHODOLOGY

In this section, we present the **SOMA** framework, which, as shown in Fig. 3, comprises an **Online Execution Workflow** and an **Offline Memory Consolidation Workflow**.

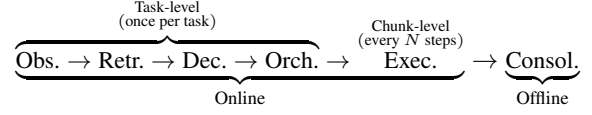
a) Online Execution Workflow: The online workflow enables robust task-level adaptation through three modules: **1. Dual-Memory RAG:** Retrieves both historical success guidance and structured failure warnings from the Dual-Memory Bank, providing task-relevant contextual grounding for decision-making.

2. Attribution-Driven LLM Orchestrator: Diagnoses execution obstacles and generates targeted interventions via attribution-guided reasoning.

3. MCP Intervention: Executes orchestrated strategies through an extensible tool interface, modulating perceptual and linguistic inputs for robust VLA reasoning.

b) Offline Memory Consolidation Workflow: Complementing online execution, the offline workflow performs **Dual-Stage Memory Consolidation**. It distills newly observed success and failure trajectories into structured memories and updates the Dual-Memory Bank, enabling continual improvement without parameter updates.

Together, these workflows form a closed-loop adaptive system.



We next describe each module in detail.

A. Dual-Memory RAG

Corresponding to Part A in Fig. 3, this module implements a retrieval-augmented Dual-Memory Bank that provides structured historical context for downstream decision-making, grounding the LLM’s reasoning in prior successes and failures. Unlike conventional RAG systems, our memory design is **dual** in two complementary aspects: (1) it explicitly separates successful and failed experiences to enable contrastive attribution; and (2) each experience undergoes a two-stage attribution process during consolidation (Section III-D), enabling continual refinement of stored signals.

We first describe the construction of the dual-memory experience bank, and then present the retrieval mechanism built upon it.

1) Dual-Memory Storage Design: We construct a dual-partition memory bank:

$$\mathcal{M} = \mathcal{M}_{succ} \cup \mathcal{M}_{fail}, \quad (1)$$

where $\mathcal{M}_{succ}$ stores successful intervention trajectories as positive guidance, and $\mathcal{M}_{fail}$ archives failed attempts together with structured attribution signals as negative evidence. This explicit separation enables contrastive reasoning between effective strategies and patterns to avoid.

This dual-partition design is empirically justified. As shown by the ablation analysis in Section III-A, using only successful or only failed experiences yields unstable or inefficient intervention planning, whereas maintaining both partitions reduces reasoning turns and variance, providing stronger decision anchoring.

Each experience entry $e_i \in \mathcal{M}$ is stored in a semi-schema format, consisting of a fixed core schema augmented with extensible auxiliary fields:

$$e_i = \left(\overbrace{E_i, T_i, S_i, D_i, V_i, O_i, \mathcal{I}_i}^{\text{fixed schema}} \right), \quad (2)$$

- E_i : multi-modal embedding of the task instance;
- T_i : natural language task description;
- $S_i \in \{0, 1\}$: success indicator;
- D_i : diagnostic annotation (non-empty for failures);
- V_i : keyframe representations of the execution video;
- O_i : structured scene as object-attribute pairs;
- $\mathcal{I}_i$: extensible structured metadata bundle.

The metadata bundle $\mathcal{I}_i$ provides a flexible tool-execution schema with parameters such as `key_frame_range`, `success_max_step`, and `rollback`. New fields can be appended as tools or reasoning signals evolve, balancing schema stability and representational flexibility.

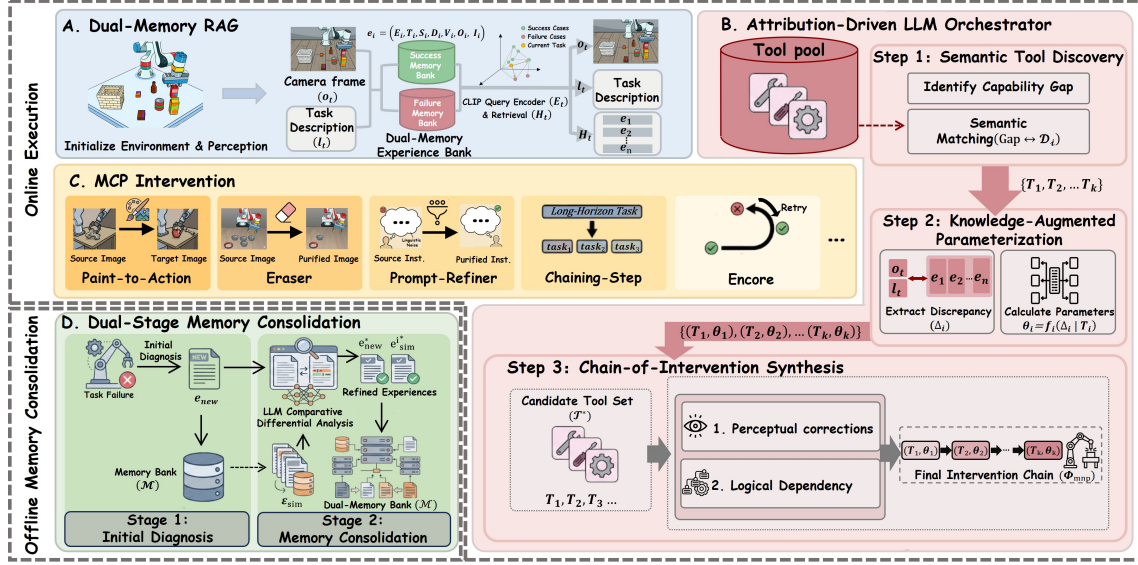


Fig. 3: **SOMA Framework.** Given the current observation and instruction, the Dual-Memory RAG module retrieves relevant experiences from a dual-memory bank. Based on this context, the LLM Orchestrator matches the MCP toolset, estimates execution parameters θ , and synthesizes an intervention chain, which is executed by the extensible MCP tools. Post-execution, memory consolidation runs asynchronously to update the Dual-Memory Bank, closing the loop for iterative refinement.

2) *Similarity-Based Retrieval Strategy:* We employ a pre-trained CLIP encoder [20] $E(\cdot, \cdot)$ to project the current task context (initial observation o_t and instruction l_t) into a joint embedding space. The retrieval is formalized as:

$$\mathcal{H}_t = \text{Top-}k\left(\text{Sim}(E(o_t, l_t), \{E_i \mid e_i \in \mathcal{M}\})\right), \quad (3)$$

$\mathcal{H}_t$ includes success exemplars and failure precedents, providing **positive guidance** and **negative counterfactuals** for attribution-driven reasoning.

The tripartite input $(\mathcal{H}_t, o_t, l_t)$ provides the necessary grounding for the LLM Orchestrator to diagnose discrepancies and sequence the appropriate intervention tools.

B. Attribution-Driven LLM Orchestrator

The LLM Orchestrator module, as illustrated in Fig. 3B, serves as the deliberative core of SOMA. In our implementation, we employ Qwen3-VL-32B as the underlying reasoning engine, leveraging its advanced multimodal understanding and logical reasoning capabilities to diagnose execution failures and orchestrate interventions via the MCP.

1) *Semantic Tool Discovery and Intent Alignment:* The LLM identifies interventions by measuring the capability gap between the current state o_t and retrieved prototypes $\mathcal{H}_t$. Given semantic tool descriptions $\{\mathcal{D}_i\}_{i=1}^N$, it performs zero-shot matching to select a candidate toolset:

$$\mathcal{T}_{1:k} = \text{Match}(\Delta_g, \{\mathcal{D}_i\}_{i=1}^N), \quad \Delta_g = \text{Gap}(o_t, \mathcal{H}_t), \quad (4)$$

selecting tools whose functions align with the diagnosed failure cause.

2) *Knowledge-Augmented Parameterization:* Following tool selection, the LLM derives execution parameters θ_i from the multi-modal context discrepancy Δ :

$$\theta_i = f(\Delta_i \mid T_i), \quad \Delta_i = \text{Gap}(o_t, l_t, \mathcal{H}_t \mid T_i), \quad (5)$$

where f denotes the mapping function for tool-specific values. Specifically, Δ characterizes the deviation of current observation o_t and instruction l_t from historical priors $\mathcal{H}_t$ across visual, semantic, and temporal dimensions. Table I summarizes the mapping from these quantitative discrepancies to the specific execution parameters θ_i for each tool.

TABLE I: Tool-specific Parameterization Mapping.

Tool	Discrepancy Δ from $\mathcal{H}_t$	θ_i
Paint	Feature distance from historical mean.	C, M, A_{mask}
Eraser	Novel geometry vs. background model.	D, D_{masks}
Prompt	Deviation from action ontologies.	$\mathcal{S}_{origin}$
Chain	Sub-event-level alignment.	$(N_e, N_b)_k$
Encore	Similarity to historical failure trajectories.	$[s_s, s_e], N_w$

C : color/texture; M : material; A_{mask} : target-object-mask; D : distractor descriptions; D_{masks} : distractor-masks; $\mathcal{S}_{origin}$: simplified goal of origin task; $N_{e,b}$: execution/buffer steps; $s_{s,e}$: start/end steps; N_w : wait duration.

The final output is a set of parameterized interventions:

$$\mathcal{T}^* = \{(T_1, \theta_1), (T_2, \theta_2), \dots, (T_k, \theta_k)\}, \quad (6)$$

3) *Chain-of-Intervention Synthesis:* The LLM then synthesizes the tools into an ordered intervention chain:

$$\Phi_{mcp} = \text{Orchestrate}(\mathcal{T}^*) \\ = \langle (T_1, \theta_1) \rightarrow (T_2, \theta_2) \rightarrow \dots \rightarrow (T_k, \theta_k) \rangle, \quad (7)$$

The orchestration follows two principles:

- 1) **Sensory-First:** Perceptual corrections (e.g., *Paint-to-Action*, *Eraser*) precede physical recovery (e.g., *Encore*) to avoid repeated failures from misperception.
- 2) **Logical Dependency:** Causal-interference removal is applied before downstream reasoning and execution.

For example, under joint visual shift and execution stagnation, the LLM generates **Paint-to-Action** $\rightarrow$ **Encore**, so perceptual rectification precedes rollback.

C. MCP Intervention

As depicted in Fig. 3C, this module executes the intervention chain Φ_{mcp} through perceptual or physical actions. We instantiate five MCP tools for common VLA failures (e.g., distribution shift and causal distraction) while preserving plug-and-play extensibility for new failure types.

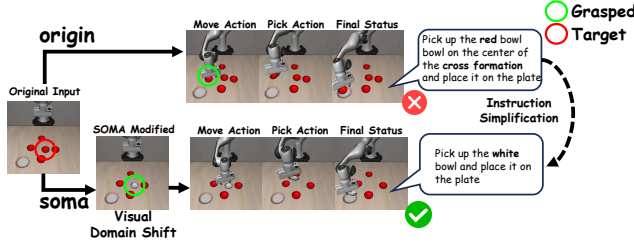


Fig. 4: **Visual Focus**. Addressing visual shift in complex manipulation environments.

1) *Paint-to-Action: Visual Domain Shift*: Paint-to-Action performs non-parametric domain adaptation:

$$o'_t = \text{Paint-to-Action}(o_t, \theta_{\text{dist}}), \quad (8)$$

Implementation is achieved through `visual_overlay` and `replace_texture`. The LLM derives θ_{dist} , SAM3 [21] segments target regions, and shift is mitigated by high-contrast masking or textures from successful trajectories (`rag.success`) (Fig. 4).

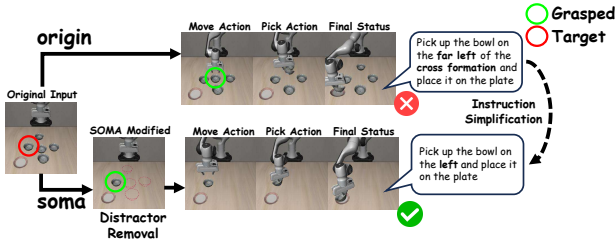


Fig. 5: **Clutter Removal**. Erasing irrelevant distractors to mitigate causal confusion.

2) *Eraser: Causal Confusion*: Eraser removes irrelevant objects via θ_{conf} :

$$o'_t = \text{Eraser}(o_t, \theta_{\text{conf}}), \quad (9)$$

Using `remove_distractor`, the LLM identifies distractors from o_t and $\mathcal{H}_t$, SAM3 [21] extracts masks, and OpenCV Telea inpainting removes them before policy inference (Fig. 5).

3) *Prompt-Refiner: Linguistic Ambiguity*: Prompt-Refiner normalizes noisy instructions:

$$l'_t = \text{Prompt-Refiner}(l_t, \theta_{\text{lang}}), \quad (10)$$

where the LLM rewrites ambiguous l_t into a policy-aligned `refined_task` using successful templates from $\mathcal{H}_t$ (e.g., colloquial commands are rewritten into concise “Pick... and place...” forms in Fig. 6).

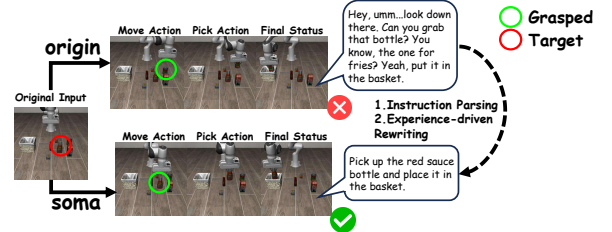


Fig. 6: **Noisy Prompt**. Rewriting instructions to reduce linguistic ambiguity.

4) *Chaining-Step: Temporal Compounding*: Chaining-Step reduces horizon length:

$$A'_{seq} = \text{Chaining-Step}(A_{seq}, \theta_{\text{chain}}), \quad (11)$$

the LLM decomposes macro sequence A_{seq} into aligned sub-tasks using $\mathcal{H}_t$, and assigns temporal bounds (notably N_e, N_b) from θ_{chain} . Replacing A_{seq} with A'_{seq} shortens execution segments and mitigates long-horizon context degradation (e.g., long tasks are split into shorter sub-tasks with “Reset & Buffer” between them in Fig. 1).

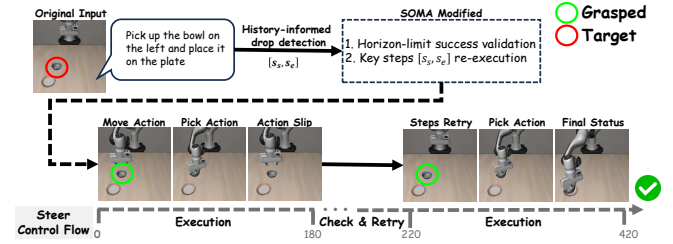


Fig. 7: **Encore**. Recovering via key-step re-execution.

5) *Encore: Execution Stagnation*: Encore resolves deadlocks via key-state recovery:

$$s'_t = \text{Encore}(s_t, \theta_{\text{stuc}}), \quad (12)$$

where θ_{stuc} (from failure precedents in $\mathcal{H}_t$) specifies retry bounds $[s_s, s_e]$ and wait time N_w . The `encore` tool resets counters and rolls back via averaged reverse trajectory; if horizon-limit validation fails, it triggers “Reset & Retry” from keyframe s_s (Fig. 7).

D. Dual-Stage Memory Consolidation

This module, illustrated in Fig. 3D, consolidates the latest execution trajectory into the Dual-Memory Bank $\mathcal{M}$ using a two-stage attribution mechanism.

1) *Stage 1: Initial Diagnosis*: Upon task termination, the LLM performs Initial Diagnosis:

$$e_{\text{new}} = \text{InitDiag}(o_t, T_{\text{raw}}), \quad (13)$$

and stores the new experience:

$$\mathcal{M} = \mathcal{M} \cup \{e_{\text{new}}\}, \quad (14)$$

TABLE II: **Performance on LIBERO-PRO.** Success rates under layout (Pos) and semantic (Task) shifts.

Level	Category	Task (Symbolic Form)	$\pi_{0.5}$ BASE	w/ SOMA
Pos	Spat.	$P.(btw(p, r), pl.)$	2%	63.0%
		$P.(center, pl.)$	0%	97.0%
		$P.(drw_{top}(cab_w), pl.)$	0%	20.2%
		$P.(next.to(p), pl.)$	0%	57.0%
		$P.(next.to(r), pl.)$	12%	71.0%
	Obj.	$P.(soup, bskt.)$	0%	35.0%
Average (Pos)			2.33%	57.2%
Task	Spat.	$P.(btw(p, r), pl.)$	0%	78.0%
		$P.(center, pl.)$	2%	82.0%
		$P.(next.to(ck.), pl.)$	2%	97.0%
		$P.(next.to(p), pl.)$	0%	73.0%
		$P.(next.to(r), pl.)$	2%	98.0%
		$P.(on(ck.), pl.)$	0%	12.0%
		$P.(on(r), pl.)$	2%	100.0%
		$P.(on(stove), pl.)$	0%	13.0%
	$P.(on(cab_w), pl.)$	0%	95.0%	
	Obj.	$P.(soup, bskt.)$	0%	16.0%
		$P.(bbq, bskt.)$	2%	32.0%
		$P.(juice, bskt.)$	2%	41.0%
		$P.(dressing, bskt.)$	0%	13.0%
		$P.(tomato, bskt.)$	0%	23.0%
Average (Task)			0.86%	55.2%

2) *Stage 2: Memory Consolidation:* Memory Consolidation retrieves the top- N similar historical experiences $\mathcal{E}_{sim} = \{e_{sim}^1, \dots, e_{sim}^N\}$ for joint cross-task refinement, formalized as:

$$\{e_{new}^*, e_{sim}^{1*}, \dots, e_{sim}^{N*}\} = \text{MemConsol}(e_{new}, \mathcal{E}_{sim}), \quad (15)$$

where each e^* is an updated experience tuple. The LLM performs batch-level differential analysis to correct historical failure descriptions and optimize correction plans using new evidence, enabling iterative self-correction of $\mathcal{M}$ rather than monotonic expansion.

IV. EXPERIMENT

We evaluate SOMA across multiple foundation models on 25 diverse tasks. We further conduct ablation studies to validate the Dual-Stage Memory Consolidation mechanism. Results show that SOMA consistently improves in-context adaptation and policy robustness across scenarios.

To improve statistical reliability under real-robot cost and trial constraints, we prioritize rigorous testing in a high-fidelity simulation environment.

A. Experimental Setup

Task Design and Evaluation Benchmarks. To evaluate robust in-context adaptation under distribution shifts, we conduct experiments across two task suites. We first adopt the **LIBERO-PRO** [7] benchmark to test robustness against Position-swap (LIBERO-PRO Pos) and Task-level (LIBERO-PRO Task) variations within Spatial and Object categories.

Furthermore, we introduce **LIBERO-SOMA**. This suite formalizes five *prevalent and critical* OOD challenge dimensions across simulated and real-world robotics. By reconfiguring assets within LIBERO scenarios [22], it provides a

controlled environment to assess an agent’s ability to trigger adaptive interventions for recurring failure modes without task-specific fine-tuning.

- **Visual Focus:** Selecting a target from five identical, novel bowls. By substituting *LIBERO-Spatial* targets with indistinguishable assets, we evaluate **referential grounding** under fine-grained visual ambiguity.
- **Clutter Removal:** Grasping targets amidst dense distractors. We augment *LIBERO-Spatial* with high-entropy object clusters to test **operational focus** and interference mitigation in crowded scenes.
- **Noisy Prompt:** Intent extraction from informal commands (e.g., “Umm...get that red squeezey thing”). By injecting linguistic noise into *LIBERO-Object* instructions, we assess **linguistic robustness** and OOD semantic mapping.
- **Execution Fragility:** Critical maneuvers where minor slips trigger failure. We curate unstable sub-tasks from *LIBERO-Object* to evaluate **operational stability** against mechanical uncertainties.
- **Task Chaining:** Long-horizon sorting of multiple objects into a basket. By sequencing atomic *LIBERO-Object* actions, this task evaluates **long-term reasoning** and **error resilience** in sequential workflows.

In-Context Initialization and Metrics. We initialize Dual-Memory RAG with a preliminary inference pass on standard LIBERO-Object and LIBERO-Spatial sets. This non-parametric “cold start” populates the memory bank with generic execution traces, without gradient-based training or exposure to evaluation-time OOD perturbations. We then run 199 randomized rollouts per task, fully shuffling object poses and task sequences to reduce bias. Performance is measured by **overall completion rates** and **intermediate success rates** for granular long-horizon analysis.

Baselines. We evaluate SOMA on diverse foundation models, including π_0 [9], $\pi_{0.5}$ [23], and SmolVLA [24]. Across OOD scenarios, SOMA consistently improves success rates, indicating better policy generalization.

B. Main Results

As shown in Fig. 8 and Table II, SOMA-augmented policies consistently surpass all baselines on both benchmarks.

1) *Results on LIBERO-SOMA: Environmental and Linguistic Variations.* Base models show sensitivity to sensory and linguistic interference. In *Visual Focus* and *Clutter Removal*, the success rates of *pi05-LIBERO* are 1.7% and 31.8%. With SOMA, these increase to 76.9% and 74.4%, indicating improved isolation of task-relevant features from environmental noise. Similarly, *smolvla-LIBERO* reaches 98.3% under *Noisy Prompt* through SOMA’s linguistic normalization. **Execution Stability and Long-Horizon Tasks.** Base models are vulnerable to *Execution Fragility*, where minor slips cause failures. SOMA adds recovery interventions, raising success rates above 87% for all models in this category. In *Task Chaining*, SOMA also improves temporal consistency under compounding errors; notably, *pi05-LIBERO* sustains 96.0% success to the final sub-task.

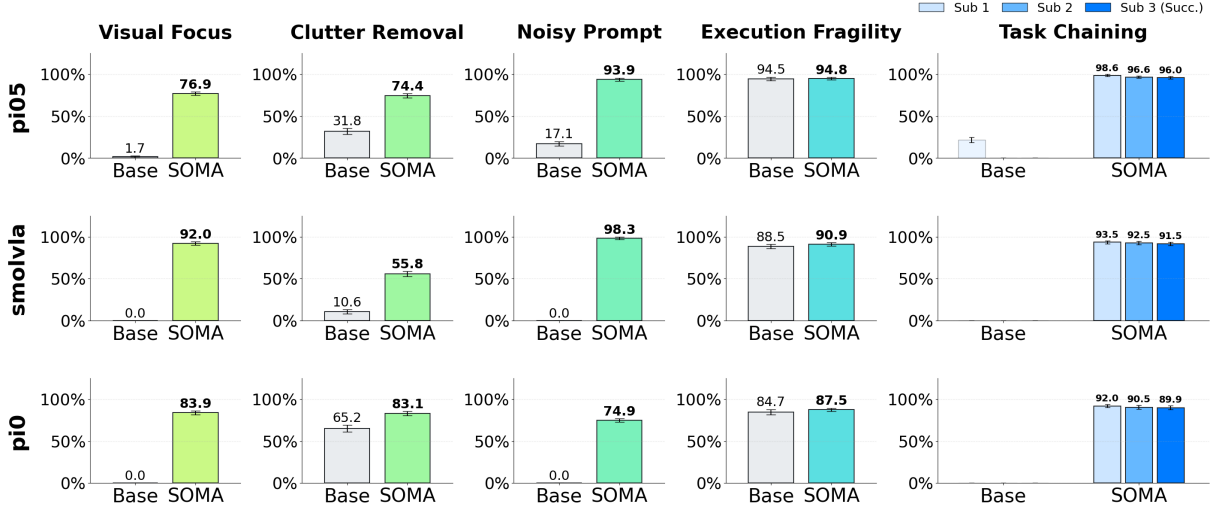


Fig. 8: **Performance on LIBERO-SOMA.** Comparison of SOMA and VLA baselines across visual, linguistic, and long-horizon challenges.

2) *Results on LIBERO-PRO: Spatial Grounding and Position Variations.* Table II details the model’s performance when facing objects-swap scenarios. The $\pi_{0.5}$ baseline struggles with these layout shifts, yielding near 0% success. In contrast, SOMA removes distractors, boosting average success rate up to 57.2%, demonstrating resilience to different positions. **Compositional Generalization and Task Perturbations.** Table II evaluates robustness to task perturbations, requiring the model to generalize across new combinations of familiar actions and objects seen during training. While the baseline exhibits near-total failure (averaging $<1\%$ success) on these instructions, SOMA demonstrates strong compositional generalization. It achieves an average success rate of 55.2%, significantly outperforming the base model which typically fails to initiate.

C. Ablation Study

With the LLM + MCP modules validated in Section IV-B, we further analyze the **Dual** design of our Dual-Memory RAG system. Following Section IV-C.1, we proceed in two steps: we first isolate the Dual-Memory Bank’s effect on execution efficiency, then evaluate the contribution of Dual-Stage Memory Consolidation to reasoning depth.

1) *Efficacy of the Dual-Memory Bank:* To verify the necessity of storing both successful and failed experiences for anchoring LLM decisions, we conduct an ablation using the Average Turns-to-Success metric [25], [26]. This metric quantifies the number of dynamic reasoning-intervention loops required to achieve an optimal plan. In our interactive setup, if an intervention fails, the agent receives sparse environmental feedback and must self-correct in subsequent turns. As illustrated in Fig. 9, we compare four memory configurations: (i) **No Memory**, (ii) **Failure Only**, (iii) **Success Only**, and (iv) **SOMA (Dual)**.

As shown in Fig. 9, the baseline without memory consistently reaches the maximum execution timeout ($Mean = 7.40$), indicating that zero-shot reasoning is insufficient for

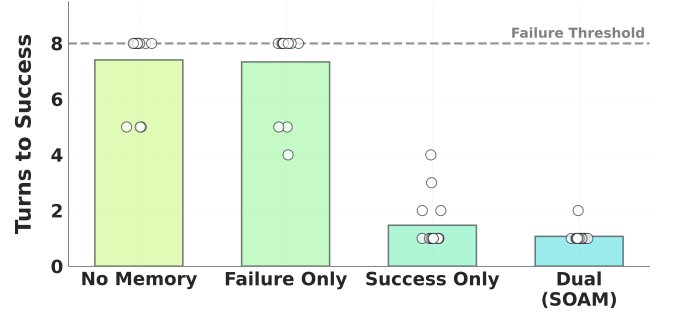


Fig. 9: **Ablation Study.** Distribution of reasoning turns required to reach the target intervention score.

OOD recovery. While *Success-Only* memory reduces the average turns to 1.47, it still exhibits significant stochastic variance. In contrast, **SOMA (Dual)** converges to a near-optimal 1.07 turns. The dual-memory bank constrains the search space for the LLM Orchestrator, enabling directed decision-making instead of stochastic exploration.

2) *Impact of Dual-Stage Memory Consolidation:* While the dual-memory bank provides the raw data, the quality of these experiences is governed by the module III-D’s iterative attribution. We evaluate this process via three configurations: **No RAG** (baseline), **Limited RAG** (single-stage online attribution), and **Rich RAG** (SOMA’s dual-stage consolidation).

Qualitative Evaluation of Reasoning Depth We use Qwen3-VL-32B to score the logical coherence of generated instructions. As shown in Fig. 10, Rich RAG consistently achieves the highest ratings across all dimensions. This gain comes from more focused reasoning trajectories that prioritize critical intervention points and reduce ineffective actions. These results suggest that higher-quality RAG data improves causal understanding and yields a more robust tool-invocation chain, consistent with the quantitative gains.

Quantitative Performance and Stability We evaluate a

TABLE III: **Ablation Study.** Comparison of reasoning logic and success rates across RAG configurations.

RAG Config	LLM Reasoning via RAG	MCP Tool	Action Result	Task Image	SR (%)
No-RAG	Ambiguity in target ID due to identical textures.	$o'_t = \text{Paint-to-Action}(o_t, \theta_{\text{dist}})$	Highlight bowl		19.2
	Potential detection errors from plate shadows.	none	—		
	Clean background, no distractors found.	none	—		
Limited-RAG	Edge proximity causes occlusion; highlight target.	$o'_t = \text{Paint-to-Action}(o_t, \theta_{\text{dist}})$	Highlight bowl		48.8
	1. Past failures suggest high mis-ID risk. 2. Focus by removing identical distractors.	$o''_t = \text{Eraser}(o'_t, \theta_{\text{conf}})$	Remove 4 dist.		
	Background replace to reduce visual noise.	$o'''_t = \text{Eraser}(o''_t, \theta_{\text{conf}})$	BG Replacement		
Rich-RAG	1. High risk from distractors via failure memory . 2. Memory dictates explicit removal of each bowl. 3. Strategic preservation of target and plate. 4. Use precise position to guide policy.	$o'_t = \text{Eraser}(o_t, \theta_{\text{conf}})$	Remove 4 dist.		60.1
	Distraction-free; no visual overlay needed.	none	—		

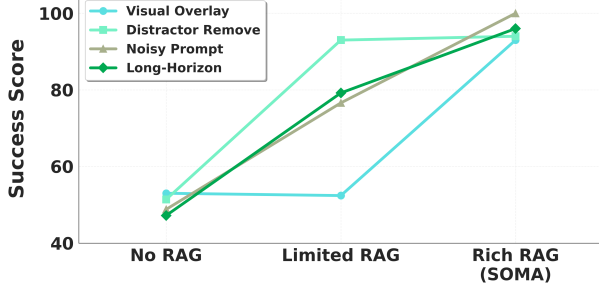


Fig. 10: **Ablation Study.** Evolution of qualitative reasoning depth scores across configurations.

“Clutter Removal” variant with $\pi_{0.5}$. As shown in Table III, success rates increase with RAG richness. No RAG achieves only 19.20%, and comparison with Section IV-B shows that LLM guidance without RAG can be counterproductive by introducing negative bias. Limited RAG improves performance to 48.80% but still relies on redundant steps (e.g., unnecessary background replacement). Rich RAG (SOMA) reaches 60.10% with a streamlined reasoning chain, showing higher strategic efficiency and precision.

V. CONCLUSIONS

This work identifies that OOD failures in VLA models arise primarily from perceptual and coordination limitations rather than motor deficiencies. To address this, we propose SOMA, a memory-augmented system that upgrades frozen VLA policies via dynamic MCP tool orchestration and a closed-loop cycle of retrieval, attribution-driven reasoning, and dual-stage memory consolidation, without any parameter fine-tuning. Evaluations across multiple VLA backbones show that SOMA consistently improves in-context adaptation and robustness under diverse OOD conditions. Future work will expand the MCP tool repository to cover a broader range of failure modes, and investigate frontier multimodal models to further refine failure attribution granularity.

REFERENCES

- [1] K. Kawaharazuka, J. Oh, J. Yamada, I. Posner, and Y. Zhu, “Vision-language-action models for robotics: A review towards real-world applications,” *IEEE Access*, vol. 13, pp. 162 467–162 504, 2025.
- [2] R. Shao, W. Li, L. Zhang, R. Zhang, Z. Liu, R. Chen, and L. Nie, “Large vlm-based vision-language-action models for robotic manipulation: A survey,” *arXiv preprint arXiv:2508.13073*, 2025.
- [3] Y. Yuan, H. Cui, Y. Chen, Z. Dong, F. Ni, L. Kou, J. Liu, P. Li, Y. Zheng, and J. Hao, “From seeing to doing: Bridging reasoning and decision for robotic manipulation,” *arXiv preprint arXiv:2505.08548*, 2025.
- [4] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of The 7th Conference on Robot Learning*, vol. 229, 2023, pp. 2165–2183.
- [5] F. Hill, S. Mokra, N. Wong, and T. Harley, “Human instruction-following with deep reinforcement learning via transfer-learning from text,” 2020.
- [6] D. Shah, B. Osiński, B. Ichter, and S. Levine, “Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action,” in *Proceedings of The 6th Conference on Robot Learning*, vol. 205, 2022, pp. 492–504.
- [7] X. Zhou, Y. Xu, G. Tie, Y. Chen, G. Zhang, D. Chu, P. Zhou, and L. Sun, “Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization,” 2025.
- [8] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu, “Libero-plus: In-depth robustness analysis of vision-language-action models,” 2025.
- [9] Physical Intelligence *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [10] M. J. Kim, K. Pertsch *et al.*, “Openvla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [11] Open X-Embodiment Collaboration *et al.*, “Open x-embodiment: Robotic learning datasets and rt-x models,” in *ICRA*, 2023.
- [12] K. Sridhar, S. Dutta, D. Jayaraman, and I. Lee, “Ricl: Adding in-context adaptability to pre-trained vision-language-action models,” *arXiv preprint arXiv:2508.02062*, 2025.
- [13] A. Sridhar, J. Pan, S. Sharma, and C. Finn, “Memer: Scaling up memory for robot control via experience retrieval,” *arXiv preprint arXiv:2510.20328*, 2025.
- [14] J. L. Liang, W. Huang *et al.*, “Code as policies: Language model programs for embodied control,” in *ICRA*, 2022.
- [15] W. Huang, C. Wang *et al.*, “Voxposer: Composable 3d value maps for robotic manipulation with language models,” in *CoRL*, 2023.
- [16] F. Liu, K. Fang, P. Abbeel, and S. Levine, “Moka: Open-world robotic manipulation through mark-based visual prompting,” in *RSS*, 2024.
- [17] X. Hou, Y. Zhao *et al.*, “Model context protocol (mcp): Landscape, security threats, and future research directions,” 2025.
- [18] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” *arXiv preprint arXiv:2303.15343*, 2023.
- [19] J. Zheng, J. Li *et al.*, “X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model,” in *Conference*, 2025.
- [20] A. Radford, J. W. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” 2021.
- [21] N. Carion, L. Gustafson, Y.-T. Hu *et al.*, “Sam 3: Segment anything with concepts,” 2025.
- [22] B. Liu, Y. Zhu, C. Gao *et al.*, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” 2023.

- [23] Physical Intelligence, K. Black, N. Brown *et al.*, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” 2025.
- [24] M. Shukor, D. Aubakirova, F. Capuano *et al.*, “Smolvla: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [25] P. Budzianowski, T.-H. Wen, B.-H. Tseng, I. Casanueva, S. Ultes, O. Ramadan, and M. Gašić, “Multiwoz – a large-scale multi-domain wizard-of-oz dataset for task-oriented dialogue modelling,” 2020.
- [26] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, F. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, and R. Pang, “Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities,” 2025.